

# Keep the Ledger, Drop the Logs: State-Keyed Context Compaction for Long-Horizon LLM Agents

Anonymous Authors

## Abstract

Long-horizon LLM agents must compress their growing interaction histories, and most compressors treat the history as text to be shortened—by recency, masking, salience or summarization. We argue that an agent’s history is better viewed as an *event log that mutates a state*, and that what compression must preserve is the *current value of each state variable*. This view exposes two distinct failure modes: *missing* state, and the more dangerous *stale* state, where the newest value is dropped but an older one survives. We formalize compression with keyed state writes and an ideal reader, and prove that (i) the compacted log—the latest record per key—is a sufficient statistic whose size is independent of the horizon and matches an information-theoretic lower bound; (ii) supersession-consistent compressors, including recency windows, are never stale, but windows forget keys by age rather than importance; and (iii) supersession-blind selection is stale with probability  $(1 - \rho)(1 - (1 - \rho)^{m-1})$  for a key with  $m$  writes when a fraction  $\rho$  of turns is kept, a law that also governs a ledger fed by an extractor of recall  $\rho$ . Building on this, we propose State-Keyed Compaction (SKC), a model-free compressor that keeps a verbatim latest-value-per-key ledger with provenance, a raw recent window and an action skeleton, plus a hybrid with relevance selection that the theory shows cannot introduce stale state. On 1,478 real SWE-agent trajectories from five LLMs, SKC retains 66%/76%/83% of agent state at 4k/8k/16k-token budgets, versus 40%/52%/66% for a recency window and 39%/43%/39% for observation masking, with a stale rate of 0.1% (1.2–2.4% for salience and random selection, reaching 9–11% on frequently rewritten keys). Its hybrid also comes within 4 points of the best baseline (and exceeds it at 16k) at preserving long-range identifiers that the agent goes on to use. A controllable benchmark confirms each theoretical prediction, including the exact stale rate under imperfect extraction, which identifies extraction quality as the key open problem for state-keyed memory.

## 1 Introduction

Large language model (LLM) agents now routinely run for tens to hundreds of steps: a software-engineering agent opens files, edits them, runs tests, reads tracebacks, and edits again [29, 33]. Every step appends an action and an observation to the context, and tool outputs dominate it: in the 1,478 SWE-agent trajectories we study, **71–89% of all context tokens are tool outputs** (Section 7.1). Even when a model’s window is large enough, long contexts cost latency and money and degrade accuracy [8, 9, 20]. Every practical agent therefore *compresses* its history—by truncating it, masking old observations, pruning “unimportant” tokens, or summarizing it with an LLM [2, 13, 19, 23].

Most of these methods treat the history as *text to be shortened*. We argue that for agents it is better viewed as an *event log that mutates a state*. An edit changes a region of a file; re-running a test replaces the previous result; a user’s correction overrides an earlier constraint. What the agent must not lose is the *current value of each state variable*, and a compressor can fail in two distinct ways: it can drop the value (*missing*), or—worse—drop the latest value while keeping an older one, so that a capable reader confidently acts on an outdated state (*stale*). Text-centric criteria such as recency or relevance do not see this distinction: a relevance scorer has no reason to prefer the newer of two test results, and a recency window has no reason to keep a constraint stated fifty steps ago.

This view suggests borrowing a mechanism that databases have used for decades: *log compaction*, which retains only the latest record per key [15, 16]. We make the idea precise and test it.

### Contributions.

1. **A state-centric formalization** of agent context compression (Section 3): histories as event logs with keyed writes, an *ideal reader* that separates information loss from reader ability, and three outcomes—*correct*, *stale*, *missing*—for each state probe.

2. **Theory** (Section 4): the compacted log is a sufficient statistic whose size is independent of the horizon and matches an information-theoretic lower bound; *supersession-consistent* compressors (which include recency windows and the compacted log) can never be stale; recency windows lose keys at a rate governed by key age rather than importance; and any compressor that keeps turns independently of supersession incurs a stale rate of  $(1 - \rho)(1 - (1 - \rho)^{m-1})$  for a key with  $m$  writes when it keeps a fraction  $\rho$  of turns—which also describes a ledger fed by an extractor with recall  $\rho$ .
3. **State-Keyed Compaction (SKC)** (Section 5): a simple, model-free compressor that keeps a latest-value-per-key ledger with verbatim values and provenance, a short raw window, and a skeleton of older actions, allocating the budget with a knapsack-ratio priority; and a hybrid, SKC+BM25, that the theory shows can add relevance-based selection without creating stale state.
4. **An empirical study** on 1,478 real SWE-agent trajectories from five LLMs (GPT-4, Claude 3 Opus, GPT-4o, Claude 3.7 Sonnet, Claude 4 Sonnet) and on a controllable benchmark, STATETRACK (Sections 6 and 7). We measure, for six compressors and budgets of 4k–16k tokens, how much agent state survives compression, how often a compressor leaves only stale state, and how well it preserves information that the agent actually goes on to use.

**Scope.** This paper measures *information sufficiency*: whether the compressed context still contains what an ideal reader would need. It does not run LLM agents end-to-end on compressed contexts. Sufficiency is a necessary condition for downstream success and isolates the compressor from the reader; the price is that our numbers are upper bounds on what a real model can recover (Section 8). All code, parsed probes and results are released.

## 2 Related Work

**Prompt and context compression.** Token-level pruning methods drop tokens with low self-information or low importance under a small LM, e.g. Selective Context [18] and the LLMLingua family [10, 11, 24]. Learned compressors map context into soft tokens [4, 7, 22], and KV-cache methods evict entries by attention statistics or position [31, 34]. These methods score text by salience; they are not designed around supersession, which our analysis shows is what produces stale state. Our BM25 baseline [26] is a turn-level, model-free representative of salience-based selection.

**Memory for LLM agents.** MemGPT pages information between a bounded context and external storage [23]; Generative Agents keep a memory stream with recency/importance/relevance retrieval and periodic reflection [25]; Reflexion stores verbal lessons [27]. MemoryBank, A-MEM and Mem0 organize long-term memories, and Mem0 explicitly applies add/update/delete operations to stored facts [5, 32, 35]. Recursive or gist summarization compresses long documents and dialogues [3, 17, 28]. Benchmarks such as LoCoMo and LongMemEval test long-term conversational memory including knowledge updates [21, 30]. Our work is complementary: rather than proposing another memory architecture, we give a formal account of *what* must be preserved within a single long-horizon episode, why common compressors fail, and an evaluation that separates missing from stale state.

**Context management for long-horizon agents.** Closest to us, Lindenbauer et al. [19] show that simply masking old observations in SWE-agent matches LLM summarization in solve rate at lower cost, and ACON [13] optimizes compression guidelines for history and observations with failure-driven feedback. MEM1 trains agents to maintain a compact internal state that replaces the history [36]. Practitioner guidance recommends “compaction” and structured note-taking for agents [2]. We provide the missing analytical lens: observation masking works because agent turns carry most state-changing information and because it is (nearly) supersession-consistent; it fails on state that only lives in observations. SKC can be seen as making the implicit state of these approaches explicit and keyed.

**State tracking and logs.** Language models’ ability to track entity states through a sequence of operations is studied by Kim and Schuster [14]. Event sourcing and log compaction in data systems keep only the latest record per key so that the state can be rebuilt from a bounded log [15, 16]; streaming lower bounds [1] motivate our counting argument.

### 3 Problem Formulation

**Histories and state.** An agent history is a sequence of turns  $h_{1:t} = (x_1, \dots, x_t)$ , each a piece of text of  $\ell(x_i)$  tokens with a role (task, agent, tool, user). Let  $\mathcal{K}$  be a set of *state keys* and  $\mathcal{V}$  a set of values. Each turn carries a (possibly empty) set of *writes*  $W(x_i) \subseteq \mathcal{K} \times \mathcal{V}$ : evidence that key  $k$  took value  $v$  at time  $i$ . The state at time  $t$  is the *fold* of the log under last-writer-wins,

$$S_t(k) = v \quad \text{where } (k, v) \in W(x_i), i = \max\{j \leq t : k \text{ written in } x_j\}, \quad (1)$$

and  $S_t(k) = \perp$  if  $k$  was never written. For a software agent, keys include the content of each edited file region, the latest output of each command, the latest view of each file window, the results of each code search, and the task itself (Section 6).

**Compressors.** A compressor  $C$  maps a history and a budget  $B$  to a context  $c = C(h_{1:t}, B)$ : a sequence of *units*, each either a (possibly truncated or masked) turn or a derived record, with total length at most  $B$  tokens (a fixed system prompt is kept and not charged).

**Ideal reader and outcomes.** To separate what a compressor removes from what a particular LLM can exploit, we evaluate with an *ideal reader*  $R^*$  that, for a key  $k$ , finds in  $c$  the evidence of the most recent write to  $k$  that is still present. Writes carry *evidence*: a set of normalized lines that identify the value (Section 6); a write is present if some unit contains at least a fraction  $\theta = 0.8$  of its evidence lines. A *state probe*  $(k, t)$  then has one of three outcomes:

- **correct**: the latest write to  $k$  is present;
- **stale**: the latest write is absent but an older write with a different value is present—a capable reader would act on outdated state;
- **missing**: no write to  $k$  is present.

We report, for each compressor and budget, the fraction of probes in each class. Stale outcomes are strictly worse than missing ones: a missing value can be re-acquired (e.g. by re-running a command) if the agent notices the gap, whereas a stale value gives no signal that anything is wrong.

**Use probes.** State probes require a definition of keys, which could favor a method built on the same keys. We therefore also use a key-free, behavior-grounded measure: at each decision point we take the identifiers (names, dotted names, paths; Appendix B) in the agent’s *next action* that already occurred earlier in the history but not in the system prompt, and ask whether each is still *available* somewhere in the compressed context. This measures whether compression removes information the agent actually went on to use.

### 4 Theory

Proofs are in Appendix A. Write  $L_t$  for the *compacted log*: for every key written up to time  $t$ , the record of its latest write.

**Proposition 1** (Sufficiency and size of the compacted log). *For every key  $k$ ,  $R^*(L_t, k) = S_t(k)$ . Its size  $|L_t| = \sum_{k \in \mathcal{K}_t} |\text{rec}(k)|$  depends only on the set  $\mathcal{K}_t$  of keys written so far and on their current values, not on the number of turns or on the number of writes. Conversely, any compressor whose context lets  $R^*$  answer every probe correctly, for all histories over  $K$  keys with values in  $\mathcal{V}$ , must use contexts of at least  $K \log_2 |\mathcal{V}|$  bits for some history.*

Thus the compacted log is optimal up to the encoding of keys and values, and the cost of a full-retention memory grows with the number of *live keys*, not with the horizon. Long agent runs are long mostly because the same keys are rewritten (tests re-run, files re-viewed); compaction removes exactly this redundancy.

**Definition 1** (Supersession consistency). A compressor is *supersession-consistent* (SC) if, for every key, the set of its writes present in the context is a suffix of its write sequence: whenever a write to  $k$  is present, every later write to  $k$  is present too.

**Proposition 2** (SC compressors are never stale). *If  $C$  is SC, every probe outcome is correct or missing. A recency window that keeps whole turns is SC; so is the compacted log with a perfect extractor; and so is any union of SC contexts.*

**Proposition 3** (Recency windows forget by age, not importance). *A recency window of  $W$  tokens answers a probe on  $k$  correctly iff the latest write to  $k$  lies within the last  $W$  tokens. If tool steps have i.i.d. lengths with mean  $\mu$  and  $k$  is written as a Poisson process with rate  $\lambda_k$  per step, then, approximating the token distance by  $\mu$  times the age in steps, the stationary probability that  $k$  is retained is  $1 - e^{-\lambda_k W / \mu}$ .*

Keys that are written rarely—the task statement, a constraint, the location of the bug found early on—are exactly the ones a window drops first, however important they are. Masking observations raises the effective  $W$  for keys written in agent turns (edits) but not for keys that live in tool outputs.

**Proposition 4** (Stale rate of supersession-blind selection). *Let a compressor keep each turn independently with probability  $\rho$  (independently of which writes it carries). For a key with  $m$  writes in distinct turns,*

$$\Pr[\text{correct}] = \rho, \quad \Pr[\text{stale}] = (1 - \rho)(1 - (1 - \rho)^{m-1}), \quad \Pr[\text{missing}] = (1 - \rho)^m. \quad (2)$$

*Hence the stale rate increases monotonically in  $m$  towards  $1 - \rho$ , and among probes for which some value is present, the fraction that is stale tends to  $1 - \rho$  as  $m \rightarrow \infty$ .*

Two consequences matter in practice. First, salience scores (BM25, token importance, attention) do not depend on supersession, so selection methods approach Equation (2): under tight budgets, *most* of the values they retain for frequently updated keys are outdated. Second, the same formula governs a ledger fed by an imperfect extractor: if each write is detected independently with recall  $r$ , the ledger behaves like independent selection with  $\rho = r$  and stops being SC. Extraction errors therefore surface as stale state, not merely as missing state—the motivation for keeping a raw recent window next to the ledger (Section 5).

**Corollary 1** (A ledger makes selection safe). *Let  $c = L' \cup c'$ , where  $L' \subseteq L_t$  is any subset of the compacted log and  $c'$  is any other context (e.g. turns chosen by relevance). A probe on  $k$  can be stale only if  $k$ 's record is not in  $L'$ . In particular, with the full compacted log, every probe is correct regardless of  $c'$ .*

The corollary licenses a hybrid: once the latest value of each key is pinned in the ledger, the remaining budget can be spent on supersession-blind but otherwise useful selection (e.g. relevance to the current step) without creating stale state.

**Proposition 5** (Budgeted ledgers). *If the compacted log exceeds the budget and key  $k$  is probed with probability  $p_k$ , maximizing the expected number of correct probes is a 0/1 knapsack with values  $p_k$  and weights  $|\text{rec}(k)|$ . Greedy selection by  $p_k/|\text{rec}(k)|$  is optimal for the fractional relaxation and, combined with the best single record, a  $\frac{1}{2}$ -approximation [6].*

## 5 State-Keyed Compaction

SKC instantiates the analysis with four parts (Algorithm 1).

**Extraction.** An extractor maps each turn to writes. For tool-using agents most writes are *structural*: they follow from the tool call itself, which the framework already has in parsed form. For SWE-agent we derive keys deterministically from actions: `edit:<file>#<region>` for the text written by an edit (regions are tracked so that only edits to overlapping lines or to text produced by an earlier edit supersede each other), `view:<file>@<window>` for file content shown to the agent, `search:<query>` and `cmd:<command>` for the outputs of searches and (normalized) shell commands. Free-text writes (e.g. a user revising a constraint) require a learned or rule-based extractor; we model its errors by its recall  $r$ .

**Ledger.** For every key, SKC keeps only the latest extracted write, rendered *verbatim* (values are capped at  $V$  tokens, keeping head and tail), with provenance (`[state] key (as of turn i)`). Verbatim values avoid the paraphrase drift of LLM summaries; provenance lets the reader order records.

---

**Algorithm 1** State-Keyed Compaction (SKC)

---

**Require:** history  $x_{1:t}$  (task  $x_1$ ), budget  $B$ , extractor  $E$ , recent turns  $R$ , value cap  $V$

- 1:  $c \leftarrow [x_1]$ ;  $c_{\text{recent}} \leftarrow$  last  $R$  turns (truncate the last turn if it alone exceeds the budget)
  - 2:  $L \leftarrow \{\}$ ; **for**  $i = 2..t$ , **for**  $(k, v) \in E(x_i)$ :  $L[k] \leftarrow (v, i)$   $\triangleright$  log compaction
  - 3:  $\mathcal{C} \leftarrow \{\text{RECORD}(k, \text{CAP}(v, V), i) : (k, (v, i)) \in L, x_i \notin c_{\text{recent}}\}$
  - 4: sort  $\mathcal{C}$  by  $p_k / |\text{RECORD}_k|$  descending; admit records while the budget allows
  - 5: **SKC**: extend backwards from  $x_{t-R}$  with whole turns while they fit, then agent turns only (tool outputs masked)
  - 6: **SKC+BM25**: instead, add older turns by BM25 score against (task, latest agent turn) while they fit
  - 7: **return**  $c \parallel$  ledger records (by provenance)  $\parallel$  skeleton  $\parallel c_{\text{recent}}$
- 

**Budget allocation.** The task statement is pinned and the last  $R$  turns are kept verbatim. Ledger records whose source turn is already in the window are skipped. Remaining records are admitted greedily by the knapsack ratio of Proposition 5, with probe probability estimated as  $p_k = w_{\text{type}(k)} \cdot \tau / (\tau + \text{age}_k)$ , where the type prior favors edits over command outputs, searches and views ( $w = 4, 2, 1.5, 1$ ) and  $\tau = 20$  steps.

**Skeleton.** Any budget left is spent extending the raw window backwards with whole turns and, once a tool output no longer fits, with agent turns only (older tool outputs replaced by a placeholder), as in observation masking. We call this variant **SKC**. A second variant, **SKC+BM25**, spends the leftover budget on the older turns with the highest BM25 score against the task and the latest agent turn, which Corollary 1 makes safe with respect to staleness.

SKC needs no model calls: extraction is a parse of the tool call, compaction is a dictionary update, and allocation is a sort. It is therefore cheap enough to run before every step, and it is idempotent: recompressing a compressed history does not compound errors, unlike recursive summarization.

## 6 Experimental Setup

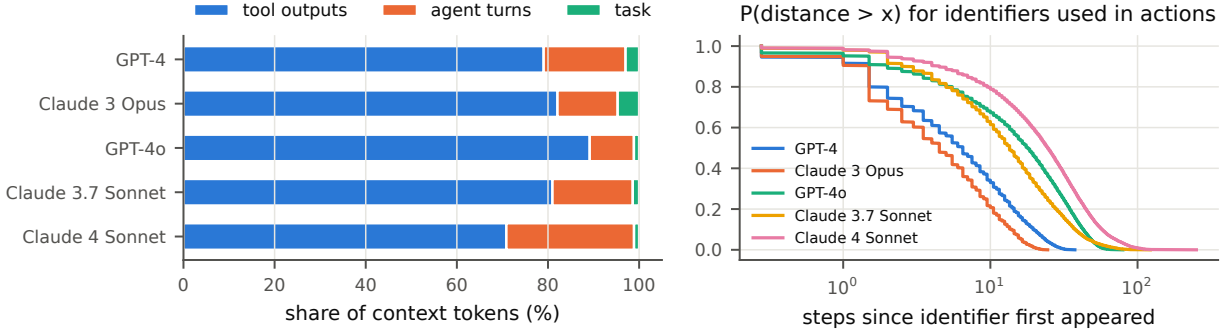
**Real trajectories.** We use the public SWE-agent trajectories submitted to the SWE-bench Lite leaderboard [12, 33] by five LLMs spanning two interface generations: GPT-4, Claude 3 Opus and GPT-4o with the SWE-agent 0.x interface, and Claude 3.7 Sonnet and Claude 4 Sonnet with SWE-agent 1.x (1,478 trajectories in total; Table 1). Each trajectory is parsed into a system turn, the task (issue text), and alternating agent and tool turns; state writes are derived from actions as described in Section 5 and Appendix B. For each trajectory and budget  $B \in \{4k, 8k, 16k\}$  tokens we evaluate up to six decision points whose uncompressed history exceeds  $B$  (7,949 / 6,227 / 4,572 decision points), probing every key written so far (125k / 119k / 114k state probes per compressor) and every eligible identifier in the next action (40k / 37k / 31k use probes).

**STATETRACK.** The synthetic benchmark generates agent-like trajectories with three key families: *pinned* constraints stated in the task and occasionally revised by the user ( $p = 0.004$  per key and step), *slow* facts rewritten rarely ( $p = 0.03$ ), and *fast* status variables rewritten often ( $p = 0.25$ ). Writes appear as `key = value` lines inside tool outputs (80%) or agent notes (20%), surrounded by log-like distractor lines (lognormal, mean 25 lines per output). Values are unique, so the ideal reader is exact. Unless stated otherwise we use 150 steps, 4 pinned, 12 slow and 6 fast keys, 30 seeds, and probe all keys at three checkpoints.

**Compressors.** We compare SKC and SKC+BM25 with five model-free baselines covering the main families used in practice (settings in Appendix C): **recency window** (FIFO truncation with the task pinned), **observation masking** [19], **truncate observations** (cap each older tool output), **BM25 selection** (salience-based selection, a turn-level stand-in for importance-pruning methods [10, 18]), and **random selection**. We do not include LLM summarization, which requires running an LLM over every prefix; see Section 8.

**Table 1:** SWE-agent trajectories used in this study (SWE-bench Lite). Tokens are counted over task, agent and tool turns of the full trajectory. “Far >10”: fraction of identifiers used in an action that first appeared more than 10 steps earlier.

| Model (interface)       | Traj. | Steps  |     | Tokens |       | Tool share | Far >10 |
|-------------------------|-------|--------|-----|--------|-------|------------|---------|
|                         |       | median | p90 | median | p90   |            |         |
| GPT-4 (0.x)             | 300   | 20     | 34  | 13.1k  | 24.7k | 79%        | 33%     |
| Claude 3 Opus (0.x)     | 300   | 18     | 23  | 7.9k   | 11.8k | 82%        | 21%     |
| GPT-4o (0.x)            | 278   | 40     | 65  | 22.6k  | 49.5k | 89%        | 67%     |
| Claude 3.7 Sonnet (1.x) | 300   | 29     | 54  | 27.3k  | 51.6k | 81%        | 61%     |
| Claude 4 Sonnet (1.x)   | 300   | 59     | 91  | 34.7k  | 56.7k | 71%        | 79%     |



**Figure 1:** Left: share of context tokens by turn type. Right: survival function of the distance (in steps) between an identifier’s first occurrence and its use in an action.

## 7 Results

### 7.1 What long agent contexts are made of

Table 1 and Figure 1 characterize the trajectories. Tool outputs make up 71–89% of all tokens. Newer models run longer and reach further back: for Claude 4 Sonnet the median trajectory has 59 steps and 35k tokens, and 79% of the identifiers its actions use first appeared more than ten steps earlier (21% for Claude 3 Opus). State writes split into command outputs (35%), file views (33%), edits (27%) and searches (5%); a median trajectory rewrites 4–7 keys at least once. Long horizons therefore come with both long-range dependencies and frequent overwrites—exactly the two conditions under which our analysis predicts that windows lose state and selection produces stale state.

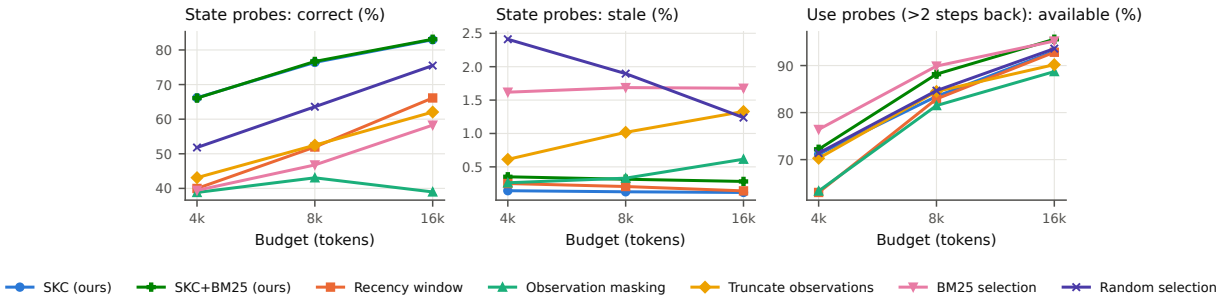
### 7.2 Main results on real trajectories

**State retention.** Table 2 and Figure 2 summarize the main comparison. At every budget SKC retains far more of the agent’s state than any baseline: 66% / 76% / 83% of state probes are correct at 4k / 8k / 16k tokens, versus 40% / 52% / 66% for the recency window and 39% / 43% / 39% for observation masking. The gain holds for each of the five models (Table 4 in the appendix): at 8k, SKC improves over the best non-random baseline by 12 (Claude 3 Opus) to 27 (Claude 4 Sonnet) points, and is within 2 points of or above every baseline for every model. Random selection is the strongest baseline on this metric because it spreads the budget over the whole history, but it pays for it in staleness (below).

**Staleness.** Supersession-blind selection leaves stale state: BM25 and random selection are stale on 1.2–2.4% of all probes, 12–24 times more than SKC (0.1%). The aggregate hides the dependence on overwrites predicted by Proposition 4: among keys written three or more times, 9–11% of BM25 and random probes are stale, i.e. 10–16% of the values these methods retain for such keys are outdated (Figure 3). Recency windows and observation masking, which are (nearly) supersession-consistent, stay at or below 2.3% even for frequently rewritten keys; residual staleness comes from truncated turns and from edits whose evidence is only partially kept.

**Table 2:** Real SWE-agent trajectories, pooled over five models. State probes: % correct and % stale (the rest are missing). Use probes: % of identifiers used by the next action that are still available; “long”: those last seen more than two steps earlier. Best in bold.

| Compressor            | State correct $\uparrow$ |             |             | State stale $\downarrow$ |            |            | Use (long) available $\uparrow$ |             |             |
|-----------------------|--------------------------|-------------|-------------|--------------------------|------------|------------|---------------------------------|-------------|-------------|
|                       | 4k                       | 8k          | 16k         | 4k                       | 8k         | 16k        | 4k                              | 8k          | 16k         |
| Recency window        | 40.0                     | 52.0        | 66.1        | 0.3                      | 0.2        | <b>0.1</b> | 63.0                            | 82.9        | 92.8        |
| Observation masking   | 38.8                     | 43.1        | 39.0        | 0.3                      | 0.3        | 0.6        | 63.3                            | 81.5        | 88.7        |
| Truncate observations | 43.1                     | 52.5        | 62.1        | 0.6                      | 1.0        | 1.3        | 70.3                            | 84.5        | 90.2        |
| BM25 selection        | 39.4                     | 46.8        | 58.3        | 1.6                      | 1.7        | 1.7        | <b>76.4</b>                     | <b>89.9</b> | 95.2        |
| Random selection      | 51.8                     | 63.6        | 75.5        | 2.4                      | 1.9        | 1.2        | 71.4                            | 84.7        | 93.6        |
| SKC (ours)            | <b>66.2</b>              | 76.4        | 83.0        | <b>0.1</b>               | <b>0.1</b> | <b>0.1</b> | 71.1                            | 83.4        | 93.1        |
| SKC+BM25 (ours)       | 66.1                     | <b>76.7</b> | <b>83.1</b> | 0.4                      | 0.3        | 0.3        | 72.2                            | 88.2        | <b>95.6</b> |



**Figure 2:** Real trajectories, pooled over five models, as a function of the budget.

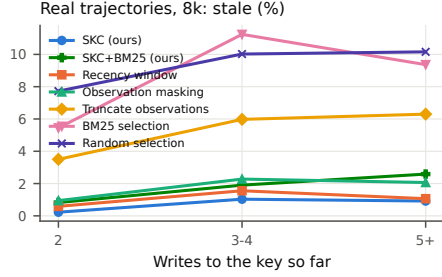
**Forgetting by age.** Figure 4 (left) breaks state probes down by the age of the key’s latest write. Recency windows and observation masking are as good as SKC for keys written in the last five steps and collapse beyond ten: for keys last written 21–40 steps ago, 14% (window) and 10% (masking) are retained, versus 70% for SKC, which degrades gracefully (58% beyond 40 steps). This is Proposition 3 in action: windows forget by age, regardless of importance.

**Per key family.** Table 3 shows where the gains come from. SKC retains 87% of command outputs and 92% of search results at 8k (window: 50% and 41%), because these are small, high-value records once the long outputs around them are dropped. Edits improve from 50–57% to 75%. File views are the exception: they are large (often a 100-line window), have the lowest priority, and are cut by the value cap, so SKC retains 52% of them, below random selection (64%). This is a deliberate trade-off of the default priority, analysed further in the ablations.

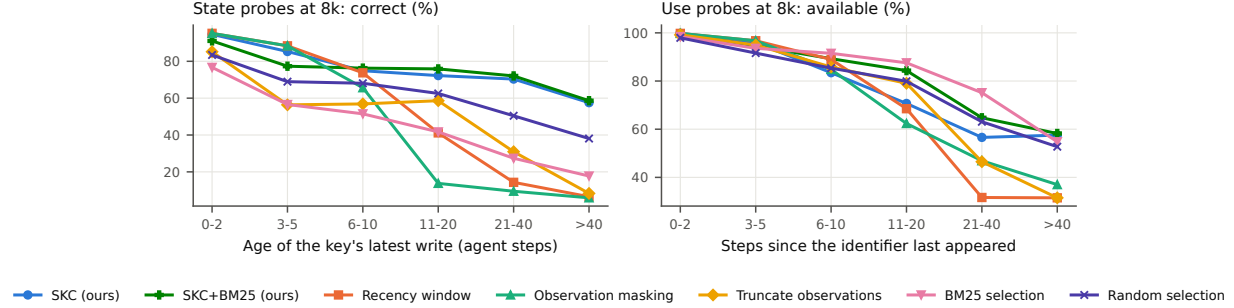
**Use probes: does compression remove what the agent uses next?** Use probes do not depend on our key definitions. For identifiers that occurred in the last two steps every method is near 100%, so we focus on long-range use probes (Table 2, right; Figure 4, right). Here the picture is different from the state probes: BM25 selection is the best single method (76% at 4k, 90% at 8k), because its query includes the latest agent turn, which is lexically close to the next action. SKC improves on the window and on masking (71% vs. 63% at 4k) but not on BM25. SKC+BM25, which spends the leftover budget on relevance, closes most of that gap (72% / 88% / 96%) while keeping SKC’s state retention and a stale rate of 0.3%, five times lower than BM25’s. In other words, the ledger and relevance-based selection are complementary: the ledger guarantees that the current value of each key is present (and, by Corollary 1, that the selected older turns cannot make it stale), and relevance recovers older context that the next step is likely to need.

### 7.3 Controlled experiments on STATETRACK

**Budget and key families.** Figure 5 separates the three families. Windows and masking retain fast keys once the budget covers a few steps but lose most slow keys (at most 38% up to 8k; window 64% and masking 42% at 16k), as Proposition 3 predicts for small  $\lambda_k$ . Selection methods retain pinned keys (their text overlaps the query) but are



**Figure 3:** Stale rate on real trajectories (8k) by the number of writes the key has received. Selection methods (BM25, random) approach the regime predicted by Proposition 4; supersession-consistent methods stay low. Keys written once cannot be stale.



**Figure 4:** Real trajectories at 8k. Left: state probes by the age of the key’s latest write. Right: use probes by the number of steps since the identifier last appeared.

stale on about 30% of all probes from 4k up. SKC reaches 96% correct at 2k and 100% from 4k, with almost no stale state. Note one subtle failure of *every* baseline: pinning the task is itself not supersession-consistent, because a user’s later revision of a pinned constraint is lost while the original stays pinned; this accounts for the 4–6% stale rate of the recency window.

**Horizon.** At a fixed 4k budget (Figure 6) the accuracy of all baselines falls as trajectories grow from 50 to 800 steps (window: 53%→31%) while their stale rates grow (BM25: 24%→48%; window: 3%→17%, from pinned constraints whose revisions fall out of the window); SKC stays at 100%, as Proposition 1 predicts when the number of keys is fixed.

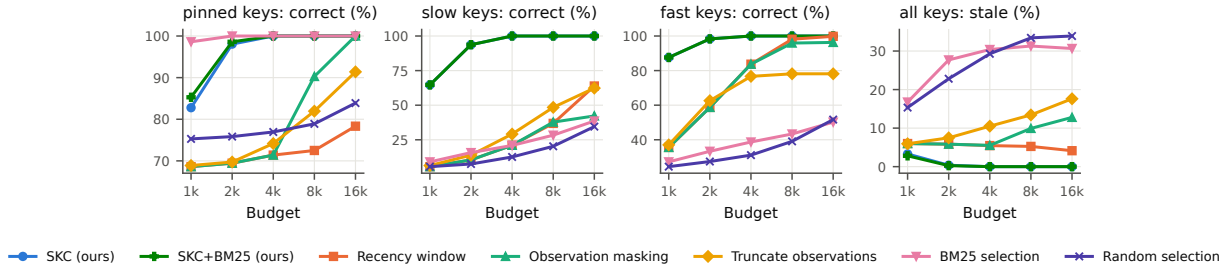
**Validating Proposition 4.** Figure 7 plots the stale rate against the number of writes  $m$  for random and BM25 selection, with Equation (2) evaluated at the empirical fraction  $\rho$  of turns kept. The prediction tracks random selection closely at all three budgets; BM25 is noisier and tends to sit above it at small budgets, because its scores are correlated with lexical overlap rather than independent. Window and SKC are never stale.

**Imperfect extraction.** Figure 8 degrades the extractor’s recall  $r$ . A ledger alone (no raw window) behaves exactly as Proposition 4 predicts with  $\rho = r$ : at  $r = 0.7$ , 25% of probes are stale. SKC’s raw recent window and backward extension hedge against missed writes, because the latest write of an actively changing key is usually recent: at 8k and  $r = 0.7$ , SKC is stale on 8% of probes and correct on 88%, against 25% and 69% for the ledger alone. SKC+BM25 is more exposed (16% stale), because its extension is supersession-blind and Corollary 1 only protects keys whose latest write the ledger actually holds. Extraction quality is therefore the critical dependency of state-keyed methods, and when it is uncertain, the leftover budget is better spent on a raw window than on relevance.

**Capacity.** When the number of keys grows at a fixed 2k budget (Figure 9), SKC degrades as the ledger stops fitting (98% at 20 keys, 64% at 100, 28% at 400), as the lower bound of Proposition 1 requires; windows and masking fall from 29% to 4%. With uniform probe probabilities and equal record sizes, every priority order is equally good (Proposition 5); we verified that recency-only and FIFO priorities match the default to within one point.

**Table 3:** Real trajectories at 8k: % correct by key family (and % stale for the mutable families). The task is pinned by every method. Probe counts: edit 24.5k, view 32.0k, cmd 47.9k, search 8.5k, task 6.2k.

| Compressor            | Correct (%) |             |             |             |             | Stale (%) |      |     |        |
|-----------------------|-------------|-------------|-------------|-------------|-------------|-----------|------|-----|--------|
|                       | task        | edit        | view        | cmd         | search      | edit      | view | cmd | search |
| Recency window        | 100         | 50.2        | 49.3        | 50.3        | 40.9        | 0.6       | 0.2  | 0.1 | 0.0    |
| Observation masking   | 100         | 51.6        | 38.9        | 35.7        | 33.7        | 0.9       | 0.5  | 0.1 | 0.0    |
| Truncate observations | 100         | 56.7        | 28.9        | 56.9        | 69.0        | 1.3       | 1.5  | 0.8 | 0.2    |
| BM25 selection        | 100         | 56.3        | 50.8        | 35.0        | 31.4        | 2.1       | 2.7  | 1.3 | 0.3    |
| Random selection      | 100         | 56.2        | <b>63.9</b> | 61.2        | 70.5        | 2.4       | 2.8  | 1.6 | 0.4    |
| SKC (ours)            | 100         | 75.4        | 52.0        | <b>87.4</b> | <b>91.6</b> | 0.2       | 0.3  | 0.0 | 0.0    |
| SKC+BM25 (ours)       | 100         | <b>75.7</b> | 53.6        | 87.0        | 91.1        | 0.2       | 0.8  | 0.1 | 0.0    |



**Figure 5:** STATETRACK (30 seeds): % correct per key family and % stale over all keys, as a function of the budget. SKC and SKC+BM25 overlap.

## 7.4 Ablations on real trajectories

Ablation results pending.

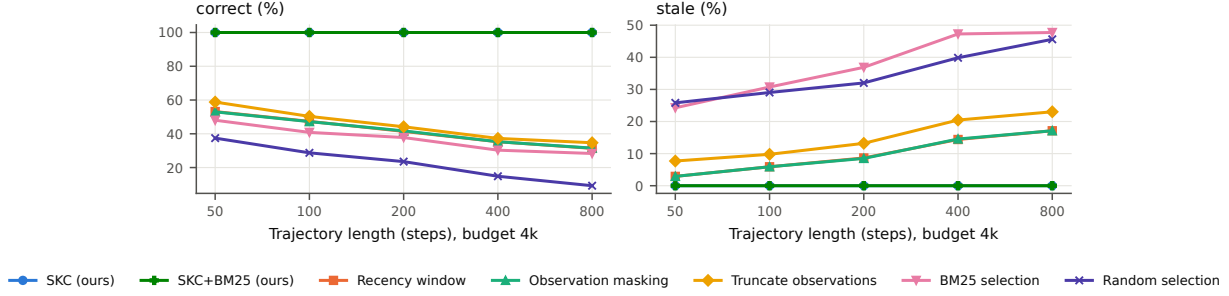
## 8 Discussion and Limitations

**Sufficiency is not success.** We measure whether information is *present* for an ideal reader. Real models may fail to use information that is present (e.g. in the middle of a long context [20]) and may recover information that is absent (by re-running a command or re-opening a file). Our numbers are therefore upper bounds on what compression permits, not predictions of solve rates. Measuring end-to-end task success with LLM agents on compressed contexts is the most important next step; this study could not call LLMs.

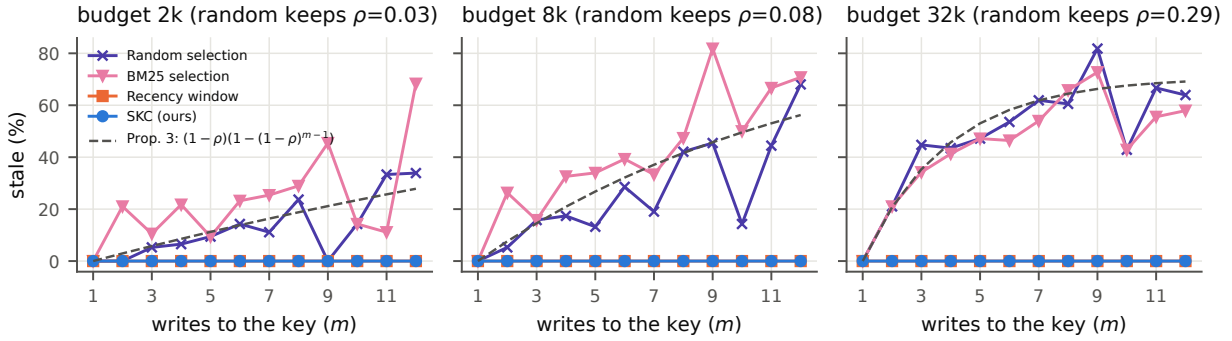
**No LLM summarization baseline.** Summarization is a widely used compressor [13, 19, 28] that we could not run. Our analysis still applies to it: a summary is superset-consistent only if the summarizer reliably replaces old values by new ones, which is precisely the extraction problem of Section 7.3, and recursive summaries can compound such errors over time. Lindenbauer et al. [19] found that observation masking matches LLM summarization in solve rate on SWE-bench, which suggests summarization does not dominate the baselines we include.

**Probe design.** State keys are derived by the same structural parser that SKC uses, which advantages SKC on state probes. We mitigate this in three ways: key-free use probes, a synthetic benchmark with exact ground truth, and noisy-extractor experiments; we also report where SKC loses (file views; long range use probes against BM25). The evidence threshold ( $\theta = 0.8$ ), the evidence sampling, and the regular-expression token counts are design choices; other choices change absolute numbers but not the qualitative ordering we observed during development.

**Domain.** Our real data come from one agent scaffold (two generations of SWE-agent) on one task family (repository-level bug fixing). Structural extraction is easiest when tools are typed; agents whose state lives mostly in free text



**Figure 6:** STATETRACK, budget 4k: accuracy and staleness vs. trajectory length. SKC and SKC+BM25 overlap at 100% correct and 0% stale.



**Figure 7:** Stale rate vs. number of writes per key ( $m$ ) in STATETRACK (200 steps, 12 fast keys). Dashed: Proposition 4 with the measured  $\rho$ .

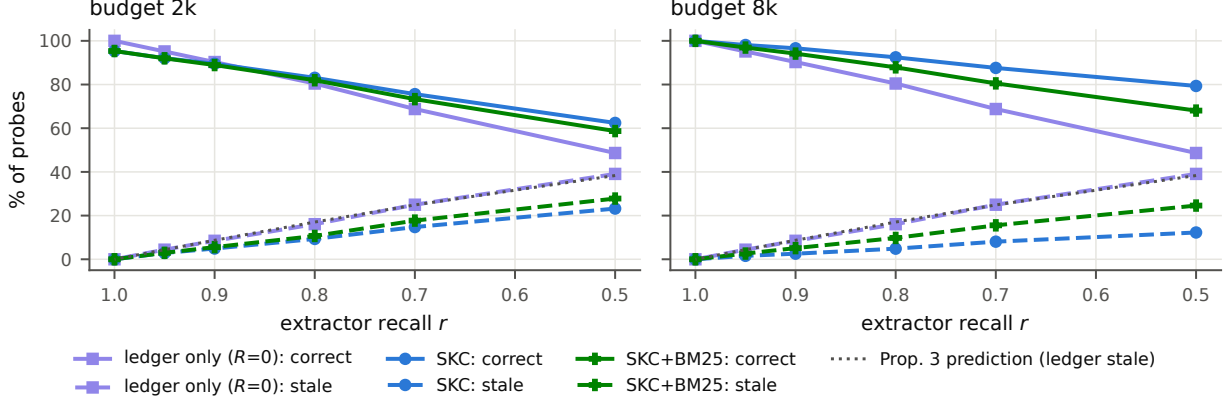
(dialogue, web browsing) need learned extractors, whose recall then governs staleness (Proposition 4).

## 9 Conclusion

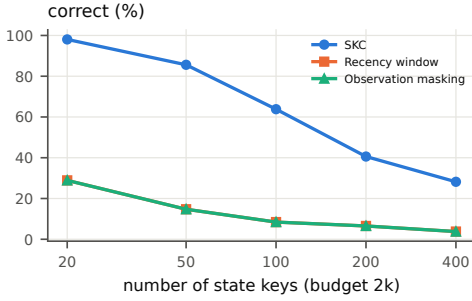
Agent histories are event logs, and the information an agent must keep is the current state those events define. Viewing compression through this lens separates two failure modes—missing and stale state—and explains when common compressors exhibit them: windows forget by age, and supersession-blind selection retains outdated values at a rate that grows with the number of overwrites. Log compaction is the natural remedy. A simple, model-free implementation, SKC, retains 66–83% of agent state on real SWE-agent trajectories at 4k–16k tokens (vs. 39–66% for windows and observation masking) with a stale rate of 0.1%, and its hybrid with relevance selection also preserves the long-range information agents go on to use. The remaining challenges are extraction for unstructured state and end-to-end validation with LLM agents.

## References

- [1] Noga Alon, Yossi Matias, and Mario Szegedy. The space complexity of approximating the frequency moments. *Journal of Computer and System Sciences*, 58(1):137–147, 1999.
- [2] Anthropic. Effective context engineering for AI agents. Anthropic Engineering Blog, 2025.
- [3] Howard Chen, Ramakanth Pasunuru, Jason Weston, and Asli Celikyilmaz. Walking down the memory maze: Beyond context limit through interactive reading. *arXiv preprint arXiv:2310.05029*, 2023.
- [4] Alexis Chevalier, Alexander Wettig, Anirudh Ajith, and Danqi Chen. Adapting language models to compress contexts. In *Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2023.



**Figure 8:** SKC variants under extractor recall  $r$  (STATETRACK, 30 seeds). Solid: correct; dashed: stale; dotted: Proposition 4 with  $\rho = r$  for the ledger alone.



**Figure 9:** Retention as the number of state keys grows at a fixed 2k budget (STATETRACK, 200 steps). Full retention requires space linear in the number of live keys (Proposition 1); SKC degrades gracefully once the ledger no longer fits, while windows retain only recently written keys.

- [5] Prateek Chhikara, Dev Khant, Saket Aryan, Taranjeet Singh, and Deshraj Yadav. Mem0: Building production-ready AI agents with scalable long-term memory. *arXiv preprint arXiv:2504.19413*, 2025.
- [6] George B. Dantzig. Discrete-variable extremum problems. *Operations Research*, 5(2):266–288, 1957.
- [7] Tao Ge, Jing Hu, Lei Wang, Xun Wang, Si-Qing Chen, and Furu Wei. In-context autoencoder for context compression in a large language model. In *International Conference on Learning Representations (ICLR)*, 2024.
- [8] Kelly Hong, Anton Troynikov, and Jeff Huber. Context rot: How increasing input tokens impacts LLM performance. Chroma Technical Report, 2025.
- [9] Cheng-Ping Hsieh, Simeng Sun, Samuel Krman, Shantanu Acharya, Dima Rekeshe, Fei Jia, Yang Zhang, and Boris Ginsburg. RULER: What’s the real context size of your long-context language models? In *Conference on Language Modeling (COLM)*, 2024.
- [10] Huiqiang Jiang, Qianhui Wu, Chin-Yew Lin, Yuqing Yang, and Lili Qiu. LLM-Lingua: Compressing prompts for accelerated inference of large language models. In *Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2023.
- [11] Huiqiang Jiang, Qianhui Wu, Xufang Luo, Dongsheng Li, Chin-Yew Lin, Yuqing Yang, and Lili Qiu. LongLLM-Lingua: Accelerating and enhancing LLMs in long context scenarios via prompt compression. In *Annual Meeting of the Association for Computational Linguistics (ACL)*, 2024.
- [12] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. SWE-bench: Can language models resolve real-world GitHub issues? In *International Conference on Learning Representations (ICLR)*, 2024.

- [13] Minki Kang, Wei-Ning Chen, Dongge Han, Huseyin A. Inan, Lukas Wutschitz, Yanzhi Chen, Robert Sim, and Saravan Rajmohan. ACON: Optimizing context compression for long-horizon LLM agents. *arXiv preprint arXiv:2510.00615*, 2025.
- [14] Najoung Kim and Sebastian Schuster. Entity tracking in language models. In *Annual Meeting of the Association for Computational Linguistics (ACL)*, 2023.
- [15] Martin Kleppmann. *Designing Data-Intensive Applications*. O’Reilly Media, 2017.
- [16] Jay Kreps, Neha Narkhede, and Jun Rao. Kafka: A distributed messaging system for log processing. In *Proceedings of the NetDB Workshop*, 2011.
- [17] Kuang-Huei Lee, Xinyun Chen, Hiroki Furuta, John Canny, and Ian Fischer. A human-inspired reading agent with gist memory of very long contexts. In *International Conference on Machine Learning (ICML)*, 2024.
- [18] Yucheng Li, Bo Dong, Frank Guerin, and Chenghua Lin. Compressing context to enhance inference efficiency of large language models. In *Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2023.
- [19] Tobias Lindenbauer, Igor Slinko, Ludwig Felder, Egor Bogomolov, and Yaroslav Zharov. The complexity trap: Simple observation masking is as efficient as LLM summarization for agent context management. *arXiv preprint arXiv:2508.21433*, 2025.
- [20] Nelson F. Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. Lost in the middle: How language models use long contexts. *Transactions of the Association for Computational Linguistics*, 12:157–173, 2024.
- [21] Adyasha Maharana, Dong-Ho Lee, Sergey Tulyakov, Mohit Bansal, Francesco Barbieri, and Yuwei Fang. Evaluating very long-term conversational memory of LLM agents. In *Annual Meeting of the Association for Computational Linguistics (ACL)*, 2024.
- [22] Jesse Mu, Xiang Lisa Li, and Noah Goodman. Learning to compress prompts with gist tokens. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.
- [23] Charles Packer, Sarah Wooders, Kevin Lin, Vivian Fang, Shishir G. Patil, Ion Stoica, and Joseph E. Gonzalez. MemGPT: Towards LLMs as operating systems. *arXiv preprint arXiv:2310.08560*, 2023.
- [24] Zhuoshi Pan, Qianhui Wu, Huiqiang Jiang, Menglin Xia, Xufang Luo, Jue Zhang, Qingwei Lin, Victor Rühle, Yuqing Yang, Chin-Yew Lin, et al. LLMingua-2: Data distillation for efficient and faithful task-agnostic prompt compression. In *Findings of the Association for Computational Linguistics: ACL*, 2024.
- [25] Joon Sung Park, Joseph O’Brien, Carrie Jun Cai, Meredith Ringel Morris, Percy Liang, and Michael S. Bernstein. Generative agents: Interactive simulacra of human behavior. In *ACM Symposium on User Interface Software and Technology (UIST)*, 2023.
- [26] Stephen Robertson and Hugo Zaragoza. The probabilistic relevance framework: BM25 and beyond. *Foundations and Trends in Information Retrieval*, 3(4):333–389, 2009.
- [27] Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.
- [28] Qingyue Wang, Liang Ding, Yanan Cao, Zhiliang Tian, Shi Wang, Dacheng Tao, and Li Guo. Recursively summarizing enables long-term dialogue memory in large language models. *arXiv preprint arXiv:2308.15022*, 2023.
- [29] Xingyao Wang, Boxuan Li, Yufan Song, Frank F. Xu, Xiangru Tang, Mingchen Zhuge, Jiayi Pan, Yueqi Song, Bowen Li, Jaskirat Singh, et al. OpenHands: An open platform for AI software developers as generalist agents. In *International Conference on Learning Representations (ICLR)*, 2025.

- [30] Di Wu, Hongwei Wang, Wenhao Yu, Yuwei Zhang, Kai-Wei Chang, and Dong Yu. LongMemEval: Benchmarking chat assistants on long-term interactive memory. In *International Conference on Learning Representations (ICLR)*, 2025.
- [31] Guangxuan Xiao, Yuandong Tian, Beidi Chen, Song Han, and Mike Lewis. Efficient streaming language models with attention sinks. In *International Conference on Learning Representations (ICLR)*, 2024.
- [32] Wujiang Xu, Zujie Liang, Kai Mei, Hang Gao, Juntao Tan, and Yongfeng Zhang. A-MEM: Agentic memory for LLM agents. *arXiv preprint arXiv:2502.12110*, 2025.
- [33] John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. SWE-agent: Agent-computer interfaces enable automated software engineering. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.
- [34] Zhenyu Zhang, Ying Sheng, Tianyi Zhou, Tianlong Chen, Lianmin Zheng, Ruisi Cai, Zhao Song, Yuandong Tian, Christopher Ré, Clark Barrett, et al. H<sub>2</sub>O: Heavy-hitter oracle for efficient generative inference of large language models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.
- [35] Wanjun Zhong, Lianghong Guo, Qiqi Gao, He Ye, and Yanlin Wang. MemoryBank: Enhancing large language models with long-term memory. In *AAAI Conference on Artificial Intelligence*, 2024.
- [36] Zijian Zhou, Ao Qu, Zhaoxuan Wu, Sunghwan Kim, Alok Prakash, Daniela Rus, Jinhua Zhao, Bryan Kian Hsiang Low, and Paul Pu Liang. MEM1: Learning to synergize memory and reasoning for efficient long-horizon agents. *arXiv preprint arXiv:2506.15841*, 2025.

## A Proofs

**Proposition 1.** By construction  $L_t$  contains, for every written key, the record of its latest write and no other record for that key, so  $R^*$  returns the value of that write, which is  $S_t(k)$  by definition. Each key contributes exactly one record whose size depends only on the key name and its current value, so  $|L_t|$  is a function of  $(\mathcal{K}, S_t)$  alone. For the lower bound, fix  $K$  keys and consider the  $|\mathcal{V}|^K$  histories that write each key once with an arbitrary value. If two such histories with different state vectors were mapped to the same context,  $R^*$  would give the same answer on both for every key and hence be wrong on at least one probe of one of them. The compressor is therefore injective on these histories, so some context has description length at least  $\log_2 |\mathcal{V}|^K = K \log_2 |\mathcal{V}|$  bits.  $\square$

**Proposition 2.** Suppose an SC context has some write to  $k$  present. The present writes of  $k$  form a non-empty suffix of its write sequence, which contains the latest write, so  $R^*$  returns the latest value (correct). Otherwise no write is present (missing). A window of whole turns keeps a suffix of turns, hence a suffix of every key’s writes. The compacted log keeps exactly the last write of each key (a suffix of length one). If  $c_1, c_2$  are SC, the present writes of  $k$  in  $c_1 \cup c_2$  are a union of two suffixes, which is a suffix.  $\square$

**Proposition 3.** The window contains exactly the turns whose starting position lies within the last  $W$  tokens, so the latest write to  $k$  is present iff it lies in that range; older writes are then absent too (no stale outcome), and if the latest write is outside the range so are all older ones (missing). For a stationary Poisson process with rate  $\lambda_k$  per step, the time since the last event (the age  $A$ ) is exponentially distributed with rate  $\lambda_k$ . Approximating the token distance by  $\mu A$ ,  $\Pr[\mu A \leq W] = 1 - e^{-\lambda_k W / \mu}$ .  $\square$

**Proposition 4.** Let the writes of  $k$  be in turns  $i_1 < \dots < i_m$ , each kept independently with probability  $\rho$ . The probe is correct iff turn  $i_m$  is kept ( $\rho$ ). It is missing iff none is kept ( $(1 - \rho)^m$ ). It is stale iff  $i_m$  is dropped and at least one of the  $m - 1$  earlier turns is kept:  $(1 - \rho)(1 - (1 - \rho)^{m-1})$  (we assume distinct values; equal values only turn stale outcomes into correct ones). This is increasing in  $m$  with limit  $1 - \rho$ . Conditioned on some write being present, the stale fraction is  $(1 - \rho)(1 - (1 - \rho)^{m-1}) / (1 - (1 - \rho)^m) \rightarrow 1 - \rho$ . For an extractor with independent recall  $r$  feeding a latest-wins ledger, the ledger record of  $k$  is the latest *detected* write; it equals the true latest write with probability  $r$ , is an older write with probability  $(1 - r)(1 - (1 - r)^{m-1})$ , and is absent otherwise, which is Equation (2) with  $\rho = r$ .  $\square$

**Corollary 1.** If  $k$ ’s record is in  $L'$ , the latest write of  $k$  is present, so  $R^*$  answers correctly whatever else  $c'$  contains, since every other present write of  $k$  is older.  $\square$

**Proposition 5.** The expected number of correct probes is  $\sum_k p_k \mathbb{E}[k \text{ kept}]$  subject to  $\sum_k |\text{rec}(k)| \mathbb{E}[k \text{ kept}] \leq B$ , a 0/1 knapsack. The ratio-greedy solution of the fractional relaxation is optimal [6]; the better of the greedy integral prefix and the single most valuable item achieves at least half of the fractional optimum.  $\square$

## B Probe and Evidence Details

**Evidence lines.** Values are split into lines; each line is normalized by removing carriage returns and editor line-number prefixes (e.g. `123:` or `123\t`) and collapsing whitespace; lines shorter than 6 characters or without alphanumerics are discarded, and duplicates are removed. For command outputs the evidence is the last 12 such lines (results, tracebacks and test summaries are printed last); for all other keys it is 12 lines sampled evenly across the value. A write is present if one unit contains at least 80% of its evidence lines. Matching is line-exact after normalization, so an edit is also present if the file view echoed after the edit shows the new lines.

**Key derivation for SWE-agent.** SWE-agent 0.x actions (`open`, `goto`, `scroll_*`, `create`, `edit a:b ...end_of_edit`, `search_dir`, `search_file`, `find_file`) and SWE-agent 1.x actions (`str_replace_editor view|create|str_replace`, `bash`) are parsed deterministically. The currently open file in 0.x is tracked from the `[File: ...]` header of observations. *Edit regions:* a line-range edit supersedes an earlier edit of the same file if their line ranges overlap; a `str_replace` supersedes an earlier edit if its `old_str` is contained in, or shares an informative line with, the text written by that edit; `create` resets the file. *View windows* are bucketed by starting line into 50-line blocks. Commands

are normalized by stripping leading `cd . . . &&` prefixes and collapsing whitespace. The task key is the issue text; it is pinned by every compressor and therefore always correct.

**Use probes.** Identifiers are maximal matches of `[A-Za-z_][\w./-]{3,}[\w]` (names, dotted names, paths without the leading slash). A use probe is an identifier that occurs in the next action, is not a stop word (interface commands, Python keywords, common shell tools), occurs in some earlier turn of the history, and does not occur in the system prompt (which every compressor keeps). We bucket use probes by the number of steps since the identifier *last* occurred; “long-range” use probes are those whose last occurrence is more than two steps back (i.e. outside the most recent turns that every compressor keeps).

**Decision points.** For each trajectory and budget we take the decision points (just before an agent turn) whose uncompressed history (excluding the system prompt) exceeds the budget, and sample up to six of them evenly. Every compressor is evaluated on the same decision points. Tokens are counted with a model-agnostic regular expression (words and punctuation); the count is of the same order as BPE token counts but not identical to any particular tokenizer, so budgets should be read as relative sizes.

## C Baseline and SKC Settings

- **Recency window:** task pinned; most recent whole turns that fit; the last turn is truncated (head and tail) if it alone exceeds the budget.
- **Observation masking** [19]: all agent turns kept; tool outputs other than the 10 most recent replaced by a placeholder; then recency window.
- **Truncate observations:** tool outputs other than the 2 most recent are cut to 200 tokens (one third head, two thirds tail); then recency window.
- **BM25 selection:** last 2 turns kept; other turns ranked by BM25 ( $k_1 = 1.2, b = 0.75$ ) against the task and the latest agent turn and added greedily; original order preserved.
- **Random selection:** last 2 turns kept; other turns added in random order while they fit.
- **SKC:**  $R = 4$  recent turns, value cap  $V = 300$  tokens (one third head, two thirds tail), type weights  $w_{\text{edit}} = 4, w_{\text{cmd}} = 2, w_{\text{search}} = 1.5, w_{\text{view}} = 1, \tau = 20$  steps; extension as described in Section 5. SKC+BM25 uses the same ledger and BM25 extension.

In STATETRACK the extractor reads the `key = value` lines that carry writes; on SWE-agent it is the structural parser above. Noisy extractors drop each write independently (deterministically by a hash of key and turn) with probability  $1 - r$ .

## D Additional Results

The per-model breakdown shows that the advantage of SKC on state probes grows with trajectory length: it is smallest for Claude 3 Opus (median 18 steps, where random selection of a short history already covers most of it) and largest for Claude 4 Sonnet (median 59 steps). On long-range use probes no method dominates across models; SKC+BM25 is best or within 4 points of the best for four of the five models.

**Table 4:** Real trajectories at 8k, per model. Top: % of state probes correct. Bottom: % of long-range use probes (last seen >2 steps earlier) available.

| Compressor                                   | GPT-4       | Claude 3 Opus | GPT-4o      | Claude 3.7 S. | Claude 4 S. |
|----------------------------------------------|-------------|---------------|-------------|---------------|-------------|
| <i>State probes: correct (%)</i>             |             |               |             |               |             |
| Recency window                               | 61.4        | 76.9          | 54.8        | 59.4          | 41.9        |
| Observation masking                          | 59.7        | 72.6          | 51.6        | 45.8          | 30.8        |
| Truncate observations                        | 61.7        | 74.6          | 58.8        | 51.6          | 45.7        |
| BM25 selection                               | 63.1        | 77.7          | 60.5        | 51.7          | 31.6        |
| Random selection                             | 75.6        | <b>91.6</b>   | 73.3        | 65.3          | 52.9        |
| SKC (ours)                                   | <b>77.0</b> | 90.0          | 81.5        | 76.2          | 73.0        |
| SKC+BM25 (ours)                              | 75.2        | 89.4          | <b>83.0</b> | <b>77.1</b>   | <b>73.3</b> |
| <i>Use probes, long range: available (%)</i> |             |               |             |               |             |
| Recency window                               | 81.1        | 87.9          | 70.0        | 84.5          | 82.5        |
| Observation masking                          | 82.4        | 89.5          | 71.9        | 78.8          | 83.8        |
| Truncate observations                        | <b>93.7</b> | 85.5          | 86.0        | 79.6          | 87.5        |
| BM25 selection                               | 91.2        | 94.4          | 79.8        | 91.6          | <b>89.1</b> |
| Random selection                             | 92.9        | <b>96.0</b>   | <b>95.6</b> | 86.0          | 82.7        |
| SKC (ours)                                   | 93.3        | 93.5          | 94.6        | 85.6          | 80.8        |
| SKC+BM25 (ours)                              | 91.6        | 93.5          | 89.7        | <b>92.1</b>   | 85.1        |